PHIẾU THỰC HÀNH — 09/10/2026

Họ tên: Nguyễn Công Trường MSSV: N24DCDT102 Lớp: D24CQDT01-N

GitHub: https://github.com/congtruong2304-oss/ctdl

Commit SHA: aacd6b1e04d64796efdd22eb1aaa5c1f635f1237

1. Mô tả một luồng ứng dụng

Ghi mục tiêu, đầu vào, đầu ra và sơ đồ truyền dữ liệu giữa các chức năng.

Mục tiêu:

- Lập kế hoạch điều phối và thực thi giao các đơn hàng từ kho (điểm 1) trong khuôn viên trường nhằm tối đa hóa tổng lợi ích trong một ngân sách thời gian giới hạn B = 8.
- Đầu vào:
    - Tệp don_hang.csv: Danh sách đơn hàng gồm mã ID, điểm giao, thời gian phục vụ và lợi ích.
    - Tệp ban_do.csv: Danh sách các cạnh vô hướng của bản đồ khuôn viên trường.
- Đầu ra: Danh sách đơn hàng được chọn tối ưu, nhật ký thực hiện giao/hoàn tác trên hàng đợi (Queue) và ngăn xếp (Stack), cùng tệp ket_qua.csv ghi nhận chi tiết đường đi, chi phí và trạng thái từng đơn.
- Sơ đồ truyền dữ liệu giữa các chức năng: Đọc 2 tệp CSV -> Chuẩn hóa danh mục (Merge Sort / Binary Search) -> Tính khoảng cách và thời gian tau (BFS) -> Chọn đơn tối ưu trong ngân sách (Quy hoạch động Knapsack) -> Điều phối giao và hoàn tác (Circular Queue / Stack) -> Xuất kết quả ra ket_qua.csv.

2. Bảng truy vết đường và thời gian

ID	Đường lượt đi	Số cạnh h	2h + phục vụ
1	1 -> 2	1	2*1 + 1 = 3
2	1 -> 2 -> 4	2	2*2 + 1 = 5
3	1 -> 2 -> 4 -> 6	3	2*3 + 1 = 7

3. Minh chứng bốn hàm

NV1 · Hoàn thiện merge_sort(a, before)

Ý tưởng/điều kiện áp dụng: Sử dụng phương pháp chia để trị. Chia dãy thành hai nửa, gọi đệ quy sắp xếp từng nửa rồi trộn hai dãy con có thứ tự lại với nhau theo hàm so sánh before. Bắt buộc giữ tính ổn định (khi hai phần tử hòa thì ưu tiên lấy phần tử ở nhánh trái trước) và không làm thay đổi mảng đầu vào.

Điều kiện dừng: Khi chiều dài mảng nhỏ hơn hoặc bằng 1, trả về bản sao của mảng.

Lời gọi: julia TH_2026_10_09_Kiem_thu.jl --phase=sort

Kết quả kiểm thử:

```
DAT pha sort: 8 phép kiểm tra.
DAT: 8 phép kiểm tra; bài sinh viên.
```

Đạt 8/8 phép kiểm tra (dãy [3, 1, 2] thành [1, 2, 3]; mảng đầu vào giữ nguyên; kiểm thử thành công mảng rỗng, mảng 1 phần tử và bảo toàn thứ tự ban đầu khi khóa bằng nhau).

Chi phí thời gian/bộ nhớ:

- Thời gian: O(n log n) trong mọi trường hợp (gồm log2(n) tầng đệ quy, mỗi tầng trộn tốn O(n)).
- Bộ nhớ: O(n) bộ nhớ phụ cho mảng đệm kết quả và O(log n) không gian ngăn xếp đệ quy.

NV2 · Hoàn thiện binary_search(a::Vector{Order}, id::Int)

Ý tưởng: Thu hẹp khoảng tìm kiếm trên mảng bằng cách so sánh khóa id với phần tử chính giữa mid = left + div(right - left, 2). Nếu bằng thì trả về bản ghi Order; nếu a[mid].id < id thì chuyển sang nửa phải (left = mid + 1); nếu a[mid].id > id thì chuyển sang nửa trái (right = mid - 1). Dừng lại khi left > right và trả về nothing.

Điều kiện áp dụng: Mảng Order bắt buộc đã được sắp xếp tăng dần theo trường id và các id là duy nhất.

Lời gọi: julia TH_2026_10_09_Kiem_thu.jl --phase=search

Kết quả kiểm thử:

```
DAT pha search: 5 phép kiểm tra.
DAT: 5 phép kiểm tra; bài sinh viên.
```

Đạt 5/5 phép kiểm tra (tìm ID 2 ở đầu dãy trả về đúng bản ghi, tìm ID 7 ở cuối dãy trả về đúng bản ghi; tìm ID 3, ID 99 hoặc tìm trên mảng rỗng trả về nothing).

Chi phí thời gian/bộ nhớ:

- Thời gian: O(log n) trong trường hợp xấu nhất; O(1) trong trường hợp tốt nhất (tìm thấy ngay tại mid đầu tiên).
- Bộ nhớ: O(1) do sử dụng giải thuật lặp và chỉ tốn vài biến chỉ số.

NV3 · Hoàn thiện bfs_routes(graph, source=1)

Ý tưởng: Dùng hàng đợi CircularQueue (nguyên tắc FIFO) để duyệt đồ thị theo từng mức khoảng cách từ đỉnh nguồn source = 1. Đánh dấu distance và nút cha parent ngay khi phát hiện đỉnh chưa thăm (distance == -1) trước khi đưa vào hàng đợi. Sau khi duyệt xong, truy vết ngược từ từng đỉnh về nguồn theo mảng parent rồi đảo ngược mảng để thu được danh sách các tuyến đường routes.

Điều kiện áp dụng: Đồ thị vô hướng không trọng số (mỗi cạnh có chi phí di chuyển bằng nhau).

Lời gọi: julia TH_2026_10_09_Kiem_thu.jl --phase=bfs

Kết quả kiểm thử:

```
DAT pha bfs: 7 phép kiểm tra.

DAT: 7 phép kiểm tra; bài sinh viên.
```

Đạt 7/7 phép kiểm tra (khoảng cách các đỉnh 1..6 từ nguồn 1 là [0, 1, 1, 2, 2, 3]; chi phí tau tương ứng cho 3 đơn là [3, 5, 7]; các đường đi hợp lệ; các đỉnh không thể tới được giữ nguyên khoảng cách -1 và đường đi rỗng).

Chi phí thời gian/bộ nhớ:

- Chi phí thời gian: O(V + E) với V là số đỉnh và E là số cạnh.
- Chi phí bộ nhớ: O(V) cho hàng đợi CircularQueue, mảng khoảng cách distance và mảng cha parent; O(V^2) khi lưu toàn bộ đường đi routes.

NV4 · Hoàn thiện knapsack(orders, costs, budget)

Ý tưởng: Sử dụng quy hoạch động xây dựng bảng table[i+1, b+1] lưu giá trị lợi ích lớn nhất khi xét i đơn hàng đầu tiên với giới hạn ngân sách b. Tại mỗi ô, nếu chi phí đơn w > b thì lấy giá trị ô hàng trên; nếu w <= b thì so sánh giữa việc bỏ qua và chọn đơn. Khi hai phương án bằng lợi ích, ưu tiên giữ phương án không lấy đơn đang xét. Sau đó truy vết ngược từ ô table[n+1, budget+1] để xác định các ID đơn hàng được chọn.

Điều kiện áp dụng: Mỗi đơn hàng chỉ được xét chọn tối đa một lần (bài toán ba lô 0/1), chi phí thời gian và ngân sách là các số nguyên không âm.

Lời gọi: julia TH_2026_10_09_Kiem_thu.jl --phase=dp

Kết quả kiểm thử:

```
DAT pha dp: 12 phép kiểm tra.
DAT: 12 phép kiểm tra; bài sinh viên.
```

Đạt toàn bộ các phép kiểm tra (giải thuật trả về đúng bộ kết quả ids, value, cost, table; kết quả tối ưu khớp hoàn toàn với kết quả từ thuật toán vét cạn brute_force; xử lý đúng trường hợp bằng lợi ích và ràng buộc ngân sách).

Chi phí thời gian/bộ nhớ:

- Chi phí thời gian: O(n * B) với n là số lượng đơn hàng và B là tổng ngân sách budget.
- Chi phí bộ nhớ: O(n * B) cho bảng quy hoạch động hai chiều table.

4. Quyết định chọn đơn

Điền ngân sách, tập chọn, tổng thời gian/lợi ích. Giải thích một ô DP và một bước truy vết; đối chiếu với vét cạn.

- Ngân sách: 8
- Tập chọn: [1, 2]
- Tổng thời gian đã dùng: 8 (Đơn 1 tốn 3, đơn 2 tốn 5)
- Tổng lợi ích tối ưu: 14 (Đơn 1 được 6, đơn 2 được 8)
- Giải thích một ô DP tiêu biểu: Xét ô tại dòng đơn 2 (chi phí w = 5, lợi ích v = 8) với mức ngân sách b = 8, tức ô table[2+1, 8+1]: nếu không lấy đơn 2 thì lợi ích lấy từ dòng trên là table[1+1, 8+1] = 6; nếu có lấy đơn 2 thì lợi ích thu được là 8 + table[1+1, (8 - 5) + 1] = 8 + table[2, 4] = 8 + 6 = 14. Do 14 lớn hơn 6 nên giá trị ô nhận giá trị cực đại là 14.
- Một bước truy vết: Bắt đầu từ ô tối ưu cuối cùng table[3+1, 8+1] = 14 với ngân sách b = 8: tại đơn 3 (chi phí w = 7), giá trị bằng hàng trên (cùng bằng 14) nên không chọn đơn 3, giữ nguyên ngân sách b = 8. Tại đơn 2 (chi phí w = 5), giá trị 14 lớn hơn ô hàng trên là 6 nên quyết định chọn đơn 2, trừ ngân sách còn lại b = 8 - 5 = 3.
- Đối chiếu với vét cạn: Giải thuật vét cạn duyệt qua toàn bộ 8 tập con đơn hàng (2 mũ 3 = 8 phương án) và cho kết quả tốt nhất là 14 với tập đơn [1, 2].

Kết quả từ bảng quy hoạch động DP hoàn toàn trùng khớp với vét cạn (DP 14 bằng Vét cạn 14).

5. Giao và hoàn tác

Trạng thái	Queue	Stack	Đã dùng	Đã nhận
Sau lập kế hoạch	[1, 2]	[ ]	0	0
Sau giao một đơn	[2]	[1]	3	6
Sau hoàn tác	[1, 2]	[ ]	0	0

6. Đầu ra và nhận xét

Đường dẫn CSV kết quả:
C:\ctdl\Bo_Thuc_hanh_Julia_2026_10_09_Sinh_vien\data\MINI001\ket_qua.csv

Link notebook: TH_2026_10_09_Pluto.jl

Lỗi đã phát hiện và cách sửa:

- Lỗi 1: Truyền nhầm tham số kiểm thử --phase=knapsack làm xuất hiện ArgumentError, đã chỉnh lại đúng cờ tiêu chuẩn là --phase=dp theo mã nguồn của bài test.
- Lỗi 2: Trong quá trình chỉnh sửa hàm binary_search vô tình làm mất định nghĩa hàm recursive_benefit gây lỗi UndefVarError, đã khôi phục lại hàm tính đệ quy để kiểm tra tính toàn vẹn khi giao và hoàn tác đơn hàng.

Một chức năng thay đổi ảnh hưởng đến chức năng sau như thế nào?

- Toàn bộ hệ thống được xây dựng theo chuỗi dữ liệu nối tiếp nhau nên chức năng trước sai sẽ kéo theo toàn bộ chức năng sau bị sai hỏng:

1. Nếu bfs_routes tính khoảng cách sai thì chi phí di chuyển costs của các đơn hàng sẽ bị tính sai.
2. Chi phí costs sai dẫn đến bảng quy hoạch động knapsack tính sai tập đơn hàng tối ưu và vượt ngân sách.
3. Tập đơn tối ưu sai sẽ nạp danh sách ID sai vào hàng đợi CircularQueue và cây BST, làm các thao tác mô phỏng deliver! (giao đơn) và undo! (hoàn tác) vi phạm các điều kiện bất biến (check_invariants) và làm sập chương trình.

Câu hỏi cuối buổi

Q1. Vì sao không được dùng dãy đã sắp giảm theo lợi ích/τ để tìm kiếm nhị phân theo ID? Nêu tiền điều kiện của hàm tìm kiếm.

- Lý do không được dùng: Tìm kiếm nhị phân dựa trên nguyên lý loại bỏ một nửa không gian tìm kiếm dựa vào quan hệ thứ tự của khóa cần tìm. Nếu dãy được sắp theo tỉ số lợi ích/tau, các khóa ID sẽ nằm lộn xộn, không còn tính đơn điệu tăng dần. Khi so sánh ID ở giữa với ID cần tìm, ta không thể biết phần tử cần tìm nằm ở nửa bên trái hay nửa bên phải.
- Tiền điều kiện của hàm binary_search: Dãy đầu vào bắt buộc phải được sắp xếp tăng dần theo trường ID và các giá trị ID phải là duy nhất.

Q2. Với đơn ID 2 tới điểm 4, phục vụ 1 và bản đồ mẫu, giải thích từng thành phần của thời gian thực hiện τ. BFS có tiếp tục cho thời gian ngắn nhất nếu các cạnh mang thời gian khác nhau không?

- Giải thích từng thành phần của tau:
    - Công thức tính chi phí: tau = 2 * (khoảng cách ngắn nhất từ 1 đến điểm giao) + thời gian phục vụ.
    - Với đơn ID 2 tới điểm 4: đường đi ngắn nhất từ kho 1 tới điểm 4 trên bản đồ mẫu là 1 -> 2 -> 4 (hoặc 1 -> 3 -> 4), gồm 2 cạnh (khoảng cách bằng 2).
    - Thời gian di chuyển 2 chiều (đi và về kho 1) là: 2 * 2 = 4.
    - Thời gian phục vụ tại điểm giao là: 1.
    - Tổng thời gian thực hiện: tau = 4 + 1 = 5.
- BFS với cạnh có trọng số khác nhau: Không. Thuật toán BFS thông thường chỉ tìm được đường đi ngắn nhất khi đồ thị không có trọng số (hoặc tất cả các cạnh có cùng trọng số bằng nhau). Nếu các cạnh có thời gian di chuyển khác nhau (đồ thị có trọng số dương), ta bắt buộc phải dùng thuật toán Dijkstra.

Q3. Giải thích hai phương án tạo ô F(2,8) và vị trí của ô đó trong table Julia. Vì sao nhánh lấy đơn phải dùng hàng trước?

- Vị trí trong table Julia: Ô F(2, 8) tương ứng với việc xét 2 đơn đầu tiên và ngân sách là 8. Do Julia đánh chỉ số mảng bắt đầu từ 1 (1-based index) và có hàng/cột 0 (cho trường hợp 0 đơn và ngân sách 0), vị trí của ô này trong mảng hai chiều là table[2 + 1, 8 + 1], tức là ô table[3, 9].
- Hai phương án tạo ô F(2, 8):

    - Phương án 1 (Không lấy đơn 2): Giữ nguyên lợi ích tốt nhất khi chỉ xét đơn 1 với cùng ngân sách 8, giá trị lấy từ ô hàng trên: table[1 + 1, 8 + 1] = 6.
    - Phương án 2 (Có lấy đơn 2): Đơn 2 có chi phí w2 = 5, lợi ích v2 = 8. Khi chọn đơn 2, ngân sách còn lại là 8 - 5 = 3. Ta cộng lợi ích của đơn 2 với lợi ích tốt nhất của các đơn trước đó với ngân sách 3: v2 + table[1 + 1, 3 + 1] = 8 + 6 = 14.
    - Giá trị ô chọn giá trị cực đại: max(6, 14) = 14.
- Vì sao nhánh lấy đơn phải dùng hàng trước: Trong bài toán ba lô 0/1, mỗi đơn hàng chỉ được phép chọn tối đa một lần. Việc tra cứu giá trị từ hàng trước (i - 1) đảm bảo rằng đơn thứ i chưa từng được chọn ở phần ngân sách còn lại đó, tránh việc một món đồ bị chọn lặp lại nhiều lần.

Q4. Sau khi giao ID 1 rồi hoàn tác ngay, FIFO, ngăn xếp, tổng thời gian và tổng lợi ích phải thay đổi thế nào? Nêu một bất biến kiểm tra việc không mất hoặc lặp đơn.

- Sự thay đổi sau khi giao rồi hoàn tác ngay:
    - Ban đầu: Queue có [1, 2], Stack rỗng [], đã dùng = 0, đã nhận = 0.
    - Sau khi giao ID 1: Queue lấy đầu ra còn [2]; ID 1 được đẩy vào Stack (thành [1]); tổng thời gian tăng thêm chi phí của đơn 1 (đã dùng = 3); tổng lợi ích tăng thêm lợi ích của đơn 1 (đã nhận = 6).
    - Sau khi hoàn tác ngay: Lấy ID 1 từ Stack ra và chèn ngược lại vào đầu Queue (dùng enqueue_front!), đưa Queue trở lại [1, 2]; Stack trở về rỗng []; tổng thời gian trừ đi chi phí đơn 1 (về lại 0); tổng lợi ích trừ đi lợi ích đơn 1 (về lại 0). Toàn bộ hệ thống trở về trạng thái y hệt lúc vừa lập kế hoạch.
- Bất biến kiểm tra không mất hoặc lặp đơn:
    - Hợp của tập các phần tử đang chờ trong hàng đợi (pending) và các phần tử đã giao (completed) luôn phải bằng đúng tập hợp các phần tử đã được chọn lúc lập kế hoạch: Set(pending) union Set(completed) == Set(selected_ids).
    - Tổng số phần tử phải được bảo toàn và không có ID nào bị trùng: length(pending) + length(completed) == length(Set(vcat(pending, completed))).

Q5. Cho biết độ phức tạp theo n,V,E,B của sắp xếp trộn, tìm kiếm nhị phân, BFS cốt lõi và DP bảng đầy đủ. Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n)?

- Độ phức tạp:
    - Sắp xếp trộn (Merge Sort): Thời gian O(n log n), bộ nhớ O(n).
    - Tìm kiếm nhị phân (Binary Search): Thời gian O(log n), bộ nhớ O(1).
    - BFS cốt lõi: Thời gian O(V + E), bộ nhớ O(V).
    - DP bảng đầy đủ (Knapsack): Thời gian O(n * B), bộ nhớ O(n * B) (với n là số đơn hàng, B là ngân sách budget).
- Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n):
    - Cây nhị phân tìm kiếm thông thường không có cơ chế tự cân bằng. Trong trường hợp xấu nhất (ví dụ: các phần tử được chèn theo thứ tự đã sắp xếp tăng dần hoặc giảm dần), cây sẽ bị suy biến hoàn toàn thành một danh sách liên kết đơn với chiều cao h = n. Khi đó, thời gian tra cứu sẽ bị thoái hóa thành O(n) chứ không còn là O(log n). Để đạt được O(log n), cây bắt buộc phải là cây cân bằng (như AVL, Cây Đỏ-Đen).